НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Звіт до комп'ютерного практикуму №3
З дисципліни «Комп'ютерне моделювання дискретно-подійних систем»
«Побудова імітаційної моделі системи з використанням формалізму моделі масового обслуговування»

Прийняв:
Викладач
Коринецький Дмитро Сергійович
«13» жовтня 2026 р.

Виконав:
Студент 4 курсу, гр. ІП-34
Скачков Іван
Олексійович

2026 р.

Зміст

Лабораторна робота №3 3
Технічне завдання 4
Виконання 7
Завдання 1 7
Лістинг файлів 9
Результат роботи 15
Висновок 16
Завдання 2 17
Формалізована модель 17
Лістинг файлів 18
Результат роботи 19
Висновок 20
Завдання 3 22
Формалізована модель 22
Лістинг файлів 23
Результат роботи 24
Висновок 26
Висновок 28
Посилання 29

Лабораторна робота №3

Тема: Побудова імітаційної моделі системи з використанням формалізму моделі масового обслуговування.

Мета: здобути практичні навички побудови універсального алгоритму імітації мереж масового обслуговування та формалізації опису реальної системи у вигляді моделі масового обслуговування.

Мова реалізації: Python 3.12, лише стандартна бібліотека. За основу взято класи з комп'ютерного практикуму №2 (Element, Create, Process, Dispose, Model), які розширено відповідно до завдань.

Технічне завдання

1. Реалізувати універсальний алгоритм імітації моделі масового обслуговування з багатоканальним обслуговуванням, з вибором маршруту за пріоритетом або за заданою ймовірністю.
2. Для наступного тексту задачі скласти формалізовану модель масового обслуговування та реалізувати її з використанням побудованого універсального алгоритму.

У банку для автомобілістів є два віконця, кожне з яких обслуговується одним касиром і має окрему під'їзну смугу. Обидві смуги розташовані поруч. Інтервали часу між прибуттям клієнтів у годину пік розподілені експоненційно з математичним сподіванням 0,5 од. часу. Тривалість обслуговування в обох касирів однакова і розподілена експоненційно з математичним сподіванням 0,3 од. часу. При рівній довжині черг, а також при відсутності черг клієнти віддають перевагу першій смузі, в усіх інших випадках вибирають більш коротку чергу. Після того як клієнт в'їхав у банк, він не може залишити його, доки не буде обслугований, проте може змінити чергу, якщо стоїть останнім і різниця в довжині черг складає не менше двох автомобілів. На кожній смузі може знаходитися не більш трьох автомобілів, тобто в банку не може бути більш восьми автомобілів разом із двома, що обслуговуються. Якщо місце перед банком заповнено, клієнт, що прибув, вважається втраченим. Початкові умови: 1) обидва касири зайняті, тривалість обслуговування для кожного нормально розподілена з математичним сподіванням 1 од. часу і середньоквадратичним відхиленням 0,3 од. часу; 2) прибуття першого клієнта заплановано на момент часу 0,1 од. часу; 3) у кожній черзі очікують по два автомобілі.

Визначити: 1) середнє завантаження кожного касира; 2) середнє число клієнтів у банку; 3) середній інтервал часу між від'їздами клієнтів від вікон; 4) середній час перебування клієнта в банку; 5) середнє число клієнтів у кожній черзі; 6) відсоток клієнтів, яким відмовлено в обслуговуванні; 7) число змін під'їзних смуг.

3. Для наступного тексту задачі скласти формалізовану модель масового обслуговування та реалізувати її з використанням побудованого універсального алгоритму.

У лікарню поступають хворі трьох типів: 1) хворі, що пройшли попереднє обстеження і направлені на лікування; 2) хворі, що бажають потрапити в лікарню, але не пройшли повністю попереднє обстеження; 3) хворі, які тільки що поступили на попереднє обстеження. Відносна частота і середній час реєстрації: тип 1 — 0,5 і 15 хвилин; тип 2 — 0,1 і 40 хвилин; тип 3 — 0,4 і 30 хвилин.

При надходженні в приймальне відділення хворий стає в чергу, якщо обидва чергових лікарі зайняті. Лікар, який звільнився, вибирає в першу чергу тих хворих, що вже пройшли попереднє обстеження. Після заповнення форм хворі типу 1 ідуть прямо в палату, а хворі типів 2 і 3 направляються в лабораторію. Троє супровідних розводять хворих по палатах; хворим не дозволяється направлятися в палату без супровідного. Якщо всі супровідні зайняті, хворі очікують їхнього звільнення в приймальному відділенні. Як тільки хворий доставлений у палату, він вважається таким, що завершив процес прийому до лікарні.

Хворі, що спрямовуються в лабораторію, не потребують супроводу. Після прибуття в лабораторію вони стають у чергу в реєстратуру, після реєстрації ідуть у кімнату очікування, де чекають виклику до одного з двох лаборантів. Після здачі аналізів хворі або повертаються в приймальне відділення (якщо їх приймають у лікарню), або залишають лікарню (якщо їм було призначено тільки попереднє обстеження). Після повернення в приймальне відділення хворий, що здав аналізи, розглядається як хворий типу 1.

Тривалість дій у хвилинах: час між прибуттями в приймальне відділення — експоненційний з математичним сподіванням 15; час слідування в палату — рівномірний від 3 до 8; час слідування з приймального відділення в лабораторію або назад — рівномірний від 2 до 5; час обслуговування в реєстратурі лабораторії — Ерланга з математичним сподіванням 4,5 і $k = 3$; час проведення аналізу — Ерланга з математичним сподіванням 4 і $k = 2$.

Визначити час, проведений хворим у системі, тобто інтервал часу від надходження до доставки в палату (для хворих типів 1 і 2) або до виходу з лабораторії (для хворих типу 3), а також інтервал між прибуттями хворих у лабораторію.

Виконання

Завдання 1

Універсальний алгоритм імітації побудовано на основі класів із попереднього практикуму. Модельний час просувається за принципом найближчої події: клас Model знаходить найменший момент tnext серед усіх елементів, накопичує статистику за пройдений інтервал, просуває час і виконує подію відповідного елемента. Модель будь-якої структури збирається з елементів Create, Process і Dispose, з'єднаних маршрутами. Діаграма класів наведена на рисунку 1, призначення класів — у таблиці 1.

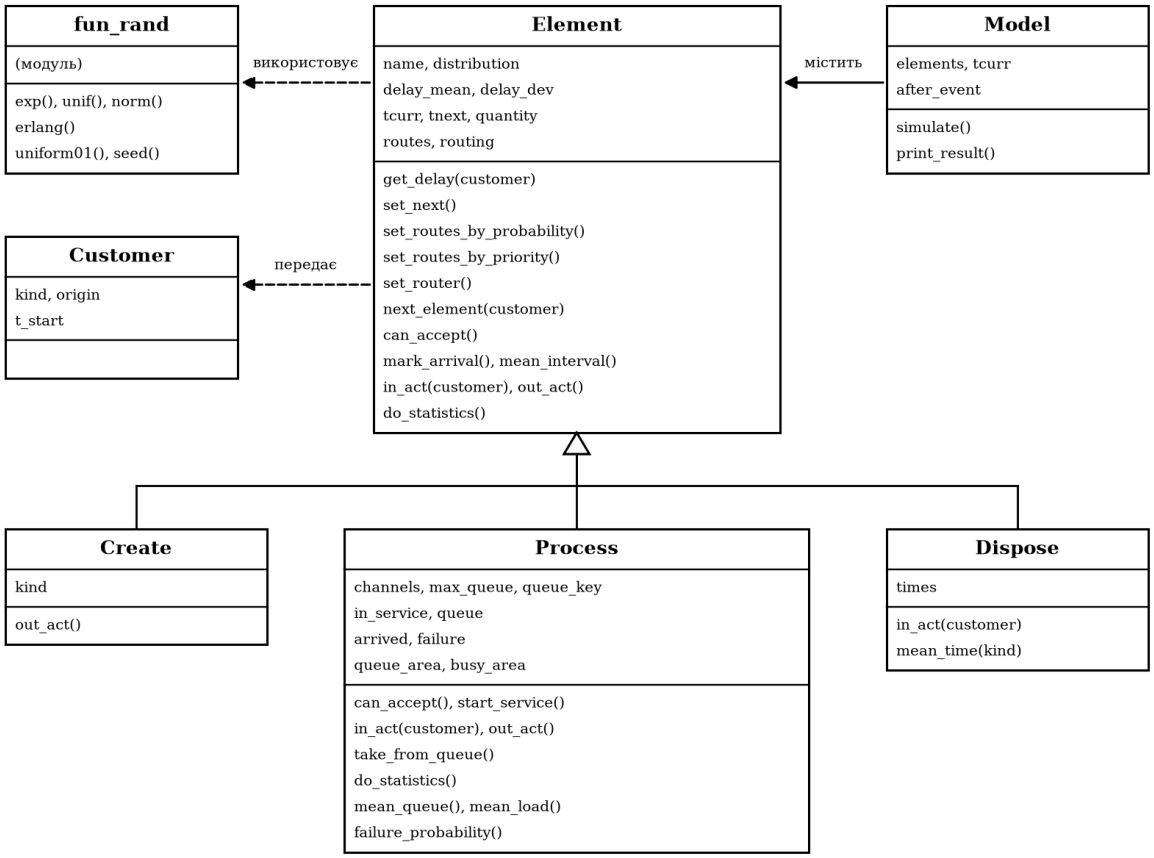


Рисунок 1 – Діаграма класів універсального алгоритму

Таблиця 1 – Класи та модулі програми

Клас (файл)	Призначення
Customer (element.py)	Вимога: поточний тип, тип при надходженні, момент надходження в модель
Element (element.py)	Базовий елемент: часова затримка, вибір наступного елемента за ймовірністю, за пріоритетом або за довільним правилом, середній інтервал між надходженнями
Create (create.py)	Джерело вимог: створює вимогу заданого типу і планує наступне надходження
Process (process.py)	Багатоканальне обслуговування: довільна кількість пристроїв, обмежена черга, пріоритет у черзі, статистика черги та завантаження
Dispose (dispose.py)	Вихід із моделі: кількість вимог і час їх перебування в системі за типами
Model (model.py)	Алгоритм імітації за принципом найближчої події; після кожної події може виконувати додаткове правило конкретної моделі
fun_rand (fun_rand.py)	Генератори: експоненційний, рівномірний, нормальний закони та закон Ерланга
theory (theory.py)	Аналітичні формули M/M/c/K для перевірки результатів імітації

Порівняно з практикумом №2 алгоритм узагальнено в таких напрямках.

- Вимога стала об’єктом. У попередній роботі вимоги лише підраховувались; тепер кожна вимога зберігає свій тип і момент надходження, що дає змогу обчислювати час перебування в системі та обслуговувати вимоги різних типів по-різному.
- Вибір маршруту трьома способами. За ймовірністю: наступний елемент вибирається випадково відповідно до заданих ймовірностей. За пріоритетом: вимога спрямовується в перший за пріоритетом елемент, який може її прийняти (має вільний пристрій або місце в черзі); якщо таких немає — у найпріоритетніший, де вона отримує відмову. За довільним правилом: функція, що за вимогою і станом моделі повертає наступний елемент.
- Багатоканальне обслуговування. Process зберігає список вимог, що обслуговуються, разом із моментами завершення; кількість пристроїв може бути необмеженою, що дає елемент чистої затримки без черги.
- Дисципліна черги. За замовчуванням вимоги обслуговуються в порядку надходження; можна задати функцію пріоритету, за якою з черги вибирається вимога.

- Часова затримка може залежати від вимоги: замість назви закону розподілу задається функція від вимоги.

Для демонстрації та перевірки побудовано модель, схему якої наведено на рисунку 2. Вимоги з CREATE спрямовуються за пріоритетом: спочатку в двоканальний PROCESS A, а якщо він заповнений — в PROCESS B. Після PROCESS B вимога з ймовірністю 0,6 залишає модель і з ймовірністю 0,4 переходить у PROCESS C.

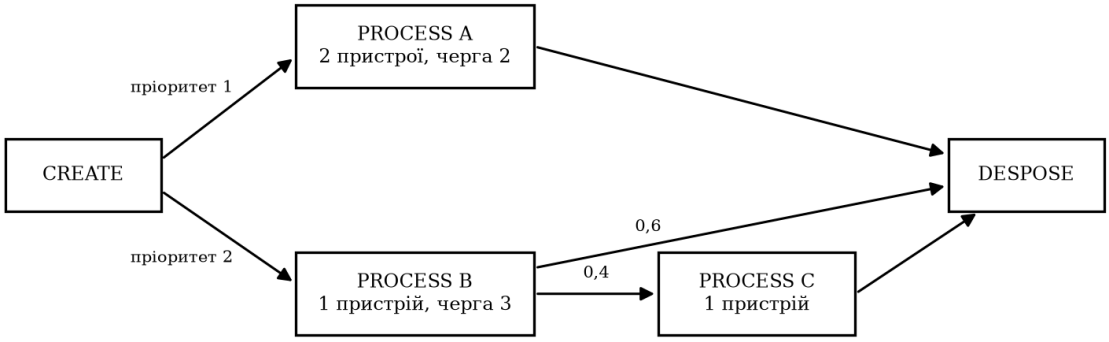


Рисунок 2 – Схема моделі для перевірки універсального алгоритму

Лістинг файлів

Файл fun_rand.py:

```
"""Генератори випадкових величин (аналог класу FunRand)."""
import math
import random


_rng = random.Random()



def seed(value):
    _rng.seed(value)



def uniform01():
    return _rng.random()



def exp(mean):
    return -mean * math.log(1 - _rng.random())



def unif(low, high):
    return low + _rng.random() * (high - low)
```

```
def norm(mean, dev):
    return mean + dev * _rng.gauss(0, 1)



def erlang(mean, k):
    """Сума k експоненційних величин із сумарним середнім mean."""
    return -mean / k * sum(math.log(1 - _rng.random()) for _ in range(k))
```

Файл element.py:

```
"""Вимога та базовий елемент моделі: часова затримка і вибір наступного елемента."""
import math


import fun_rand



class Customer:
    def __init__(self, kind, t_start):
        self.kind = kind  # поточний тип, може змінюватись у моделі
        self.origin = kind  # тип при надходженні
        self.t_start = t_start



class Element:
    def __init__(self, name, delay_mean=1.0, delay_dev=0.0, distribution="exp"):
        self.name = name
        self.delay_mean = delay_mean
        self.delay_dev = delay_dev
        self.distribution = distribution  # назва закону або функція(вимога) -> затримка
        self.tcurr = 0.0
        self.tnext = math.inf
        self.quantity = 0
        self.routes = []
        self.routing = "probability"
        self.last_in = None
        self.gap_sum = 0.0
        self.gaps = 0


    def get_delay(self, customer=None):
        if callable(self.distribution):
            return self.distribution(customer)
        if self.distribution == "exp":
            return fun_rand.exp(self.delay_mean)
        if self.distribution == "norm":
            return fun_rand.norm(self.delay_mean, self.delay_dev)
        if self.distribution == "unif":
            return fun_rand.unif(self.delay_mean, self.delay_dev)
        if self.distribution == "erlang":
            return fun_rand.erlang(self.delay_mean, self.delay_dev)
        return self.delay_mean


    def set_next(self, element):
        self.set_routes_by_probability([(element, 1.0)])


    def set_routes_by_probability(self, routes):
        """routes — список (елемент, ймовірність)."""
        self.routes, self.routing = routes, "probability"


    def set_routes_by_priority(self, elements):
        """elements — елементи в порядку спадання пріоритету."""
        self.routes, self.routing = elements, "priority"


    def set_router(self, function):
        """function(вимога) -> елемент: довільне правило вибору маршруту."""
        self.routing = function


    def next_element(self, customer):
        if callable(self.routing):
```

```
            return self.routing(customer)
        if self.routing == "priority":
            for element in self.routes:  # перший за пріоритетом, який може прийняти вимогу
                if element.can_accept():
                    return element
            return self.routes[0] if self.routes else None
        r, cumulative = fun_rand.uniform01(), 0.0
        for element, probability in self.routes:
            cumulative += probability
            if r < cumulative:
                return element
        return None


    def can_accept(self):
        return True


    def mark_arrival(self):
        if self.last_in is not None:
            self.gap_sum += self.tcurr - self.last_in
            self.gaps += 1
        self.last_in = self.tcurr


    def mean_interval(self):
        """Середній інтервал між надходженнями вимог в елемент."""
        return self.gap_sum / self.gaps if self.gaps else 0.0


    def in_act(self, customer):
        pass


    def out_act(self):
        self.quantity += 1


    def do_statistics(self, delta):
        pass


    def print_info(self):
        print(f"  {self.name}: quantity = {self.quantity}, tnext = {self.tnext:.4f}")


    def print_result(self, time):
        print(f"{self.name}: quantity = {self.quantity}")
```

Файл create.py:

```
"""CREATE: джерело вимог."""
from element import Customer, Element



class Create(Element):
    def __init__(self, name, delay_mean, delay_dev=0.0, distribution="exp", kind=None):
        super().__init__(name, delay_mean, delay_dev, distribution)
        self.tnext = 0.0
        self.kind = kind  # функція, що повертає тип нової вимоги


    def out_act(self):
        super().out_act()
        self.tnext = self.tcurr + self.get_delay()
        customer = Customer(self.kind() if self.kind else 0, self.tcurr)
        target = self.next_element(customer)
        if target:
            target.in_act(customer)
```

Файл process.py:

```
"""PROCESS: багатоканальне обслуговування з обмеженою чергою і пріоритетом у черзі."""
import math


from element import Element
```

```
class Process(Element):
    def __init__(self, name, delay_mean=1.0, delay_dev=0.0, distribution="exp",
                 channels=1, max_queue=math.inf, queue_key=None):
        super().__init__(name, delay_mean, delay_dev, distribution)
        self.channels = channels  # math.inf — затримка без обмеження кількості пристроїв
        self.max_queue = max_queue
        self.queue_key = queue_key  # функція(вимога) -> пріоритет у черзі, менше — раніше
        self.in_service = []  # пари (момент завершення, вимога)
        self.queue = []
        self.arrived = 0
        self.failure = 0
        self.queue_area = 0.0
        self.busy_area = 0.0


    @property
    def busy(self):
        return len(self.in_service)


    def can_accept(self):
        return self.busy < self.channels or len(self.queue) < self.max_queue


    def start_service(self, customer, delay=None):
        delay = self.get_delay(customer) if delay is None else delay
        self.in_service.append((self.tcurr + delay, customer))
        self.tnext = min(t for t, _ in self.in_service)


    def in_act(self, customer):
        self.mark_arrival()
        self.arrived += 1
        if self.busy < self.channels:
            self.start_service(customer)
        elif len(self.queue) < self.max_queue:
            self.queue.append(customer)
        else:
            self.failure += 1


    def take_from_queue(self):
        if self.queue_key is None:
            return self.queue.pop(0)
        index = min(range(len(self.queue)), key=lambda i: (self.queue_key(self.queue[i]), i))
        return self.queue.pop(index)


    def out_act(self):
        for item in [x for x in self.in_service if x[0] == self.tcurr]:
            self.in_service.remove(item)
            super().out_act()
            if self.queue:
                self.start_service(self.take_from_queue())
            self.tnext = min((t for t, _ in self.in_service), default=math.inf)
            customer = item[1]
            target = self.next_element(customer)
            if target:
                target.in_act(customer)


    def do_statistics(self, delta):
        self.queue_area += len(self.queue) * delta
        self.busy_area += self.busy * delta


    def mean_queue(self, time):
        return self.queue_area / time


    def mean_load(self, time):
        """Частка зайнятих пристроїв; для необмеженої кількості — середнє число зайнятих."""
        return self.busy_area / time / (1 if self.channels == math.inf else self.channels)


    def failure_probability(self):
        return self.failure / self.arrived if self.arrived else 0.0
```

```
    def print_info(self):
        print(f"  {self.name}: busy = {self.busy}/{self.channels}, queue = {len(self.queue)}, "
              f"quantity = {self.quantity}, failure = {self.failure}, tnext = {self.tnext:.4f}")


    def print_result(self, time):
        print(f"{self.name}: arrived = {self.arrived}, quantity = {self.quantity}, "
              f"failure = {self.failure}")
        print(f"    mean queue = {self.mean_queue(time):.4f}, "
              f"mean load = {self.mean_load(time):.4f}, "
              f"failure probability = {self.failure_probability():.4f}")
```

Файл dispose.py:

```
"""DESPOSE: вихід вимог із моделі та час їх перебування в системі."""
from element import Element



class Dispose(Element):
    def __init__(self, name):
        super().__init__(name)
        self.times = {}  # тип вимоги -> [кількість, сумарний час у системі]


    def in_act(self, customer):
        self.mark_arrival()
        self.quantity += 1
        stat = self.times.setdefault(customer.origin, [0, 0.0])
        stat[0] += 1
        stat[1] += self.tcurr - customer.t_start


    def mean_time(self, kind=None):
        """Середній час у системі для вимог типу kind (None — для всіх)."""
        stats = [s for k, s in self.times.items() if kind is None or k == kind]
        count = sum(s[0] for s in stats)
        return sum(s[1] for s in stats) / count if count else 0.0
```

Файл model.py:

```
"""Model: універсальний алгоритм імітації за принципом найближчої події."""



class Model:
    def __init__(self, elements, after_event=None):
        self.elements = elements
        self.after_event = after_event  # правило конкретної моделі, виконується після кожної події
        self.tcurr = 0.0


    def simulate(self, time, verbose_events=0):
        """verbose_events — скільки перших подій вивести у протокол."""
        step = 0
        while True:
            tnext = min(e.tnext for e in self.elements)
            if tnext > time:  # статистика збирається рівно на інтервалі [0; time]
                tnext = time
            for e in self.elements:
                e.do_statistics(tnext - self.tcurr)
            self.tcurr = tnext
            if self.tcurr >= time:
                break
            for e in self.elements:
                e.tcurr = self.tcurr
            for e in self.elements:
                if e.tnext == self.tcurr:
                    if step < verbose_events:
                        print(f"\nПодія в {e.name}, t = {self.tcurr:.4f}")
                    e.out_act()
            if self.after_event:
                self.after_event()
            if step < verbose_events:
                for e in self.elements:
                    e.print_info()
```

```
            step += 1

    def print_result(self):
        print(f"\n------------- РЕЗУЛЬТАТИ (T = {self.tcurr:g}) -------------")
        for e in self.elements:
            e.print_result(self.tcurr)
```

Файл theory.py:

```
"""Аналітичні характеристики СМО M/M/c/K для перевірки результатів імітації."""



def mmck(arrival_mean, service_mean, channels=1, max_queue=1000):
    """Повертає (ймовірність відмови, середня черга, середнє завантаження пристрою).

    arrival_mean — середній інтервал між вимогами, service_mean — середній час обслуговування.
    Нескінченна черга наближається великим max_queue (при завантаженні < 1).
    """
    a = service_mean / arrival_mean
    weights = [1.0]
    for n in range(1, channels + max_queue + 1):
        weights.append(weights[-1] * a / min(n, channels))
    total = sum(weights)
    p = [w / total for w in weights]
    p_fail = p[-1]
    mean_queue = sum((n - channels) * p[n] for n in range(channels + 1, len(p)))
    mean_load = sum(min(n, channels) * p[n] for n in range(len(p))) / channels
    return p_fail, mean_queue, mean_load
```

Файл task1_universal.py:

```
"""Завдання 1: універсальний алгоритм — багатоканальність, маршрут за пріоритетом і ймовірністю.

CREATE -> за пріоритетом: PROCESS A (2 пристрої, черга 2) -> DESPOSE
                          PROCESS B (1 пристрій, черга 3) -> за ймовірністю:
                                DESPOSE (0.6) або PROCESS C (0.4) -> DESPOSE
"""
import fun_rand
from create import Create
from dispose import Dispose
from model import Model
from process import Process
from theory import mmck

DELAY_CREATE, DELAY_A, P_TO_C, TIME = 1.0, 1.5, 0.4, 100_000



def run(verbose_events=0):
    fun_rand.seed(1)
    create = Create("CREATE", DELAY_CREATE)
    a = Process("PROCESS A", DELAY_A, channels=2, max_queue=2)
    b = Process("PROCESS B", 1.0, channels=1, max_queue=3)
    c = Process("PROCESS C", 2.0)
    dispose = Dispose("DESPOSE")

    create.set_routes_by_priority([a, b])
    a.set_next(dispose)
    b.set_routes_by_probability([(dispose, 1 - P_TO_C), (c, P_TO_C)])
    c.set_next(dispose)

    model = Model([create, a, b, c, dispose])
    model.simulate(TIME, verbose_events)
    return model, create, (a, b, c), dispose



def main():
    model, create, (a, b, c), dispose = run(verbose_events=6)
```

```
    model.print_result()

    # вимоги, які PROCESS A не може прийняти, ідуть в обхід: для нього це M/M/2/4 з втратами
    p_block, mean_queue, mean_load = mmck(DELAY_CREATE, DELAY_A, 2, 2)
    bypassed = (b.arrived + a.failure) / create.quantity
    print("\nМаршрут за пріоритетом, PROCESS A (імітація / теорія M/M/2/4):")
    print(f"  частка вимог, не прийнятих у PROCESS A: {bypassed:.4f} / {p_block:.4f}")
    print(f"  середнє завантаження:                   {a.mean_load(TIME):.4f} / {mean_load:.4f}")
    print(f"  середня довжина черги:                  {a.mean_queue(TIME):.4f} / {mean_queue:.4f}")

    print("\nМаршрут за ймовірністю (імітація / задана ймовірність):")
    print(f"  PROCESS B -> PROCESS C: {c.arrived / b.quantity:.4f} / {P_TO_C}")

    in_system = sum(len(p.queue) + p.busy for p in (a, b, c))
    lost = a.failure + b.failure + c.failure
    balance = create.quantity == dispose.quantity + lost + in_system
    print(f"\nБаланс: створено {create.quantity} = вийшло {dispose.quantity} + втрачено {lost} "
          f"+ у системі {in_system}: {'так' if balance else 'НІ'}")



if __name__ == "__main__":
    main()
```

Результат роботи

```
Подія в CREATE, t = 0.0000
  CREATE: quantity = 1, tnext = 0.1443
  PROCESS A: busy = 1/2, queue = 0, quantity = 0, failure = 0, tnext = 2.8202
  PROCESS B: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  PROCESS C: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  DESPOSE: quantity = 0, tnext = inf


Подія в CREATE, t = 0.1443
  CREATE: quantity = 2, tnext = 1.5873
  PROCESS A: busy = 2/2, queue = 0, quantity = 0, failure = 0, tnext = 0.5860
  PROCESS B: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  PROCESS C: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  DESPOSE: quantity = 0, tnext = inf


Подія в PROCESS A, t = 0.5860
  CREATE: quantity = 2, tnext = 1.5873
  PROCESS A: busy = 1/2, queue = 0, quantity = 1, failure = 0, tnext = 2.8202
  PROCESS B: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  PROCESS C: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  DESPOSE: quantity = 1, tnext = inf


Подія в CREATE, t = 1.5873
  CREATE: quantity = 3, tnext = 2.1842
  PROCESS A: busy = 2/2, queue = 0, quantity = 1, failure = 0, tnext = 2.8202
  PROCESS B: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  PROCESS C: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  DESPOSE: quantity = 1, tnext = inf


Подія в CREATE, t = 2.1842
  CREATE: quantity = 4, tnext = 3.7388
  PROCESS A: busy = 2/2, queue = 1, quantity = 1, failure = 0, tnext = 2.8202
  PROCESS B: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  PROCESS C: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  DESPOSE: quantity = 1, tnext = inf


Подія в PROCESS A, t = 2.8202
  CREATE: quantity = 4, tnext = 3.7388
  PROCESS A: busy = 2/2, queue = 0, quantity = 2, failure = 0, tnext = 2.9681
  PROCESS B: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  PROCESS C: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  DESPOSE: quantity = 2, tnext = inf


------------ РЕЗУЛЬТАТИ (T = 100000) -------------
CREATE: quantity = 100027
```

```
PROCESS A: arrived = 87956, quantity = 87610, failure = 344
    mean queue = 0.4150, mean load = 0.6568, failure probability = 0.0039
PROCESS B: arrived = 12071, quantity = 12071, failure = 0
    mean queue = 0.0748, mean load = 0.1204, failure probability = 0.0000
PROCESS C: arrived = 4718, quantity = 4718, failure = 0
    mean queue = 0.0257, mean load = 0.0952, failure probability = 0.0000
DESPOSE: quantity = 99681


Маршрут за пріоритетом, PROCESS A (імітація / теорія M/M/2/4):
  частка вимог, не прийнятих у PROCESS A: 0.1241 / 0.1240
  середнє завантаження:                   0.6568 / 0.6570
  середня довжина черги:                  0.4150 / 0.4135


Маршрут за ймовірністю (імітація / задана ймовірність):
  PROCESS B -> PROCESS C: 0.3909 / 0.4


Баланс: створено 100027 = вийшло 99681 + втрачено 344 + у системі 2: так
```

Таблиця 2 – Перевірка універсального алгоритму (T = 100000)

Показник	Імітація	Очікуване значення
Частка вимог, не прийнятих у PROCESS A	0,1241	0,1240
Середнє завантаження пристроїв PROCESS A	0,6568	0,6570
Середня довжина черги PROCESS A	0,4150	0,4135
Частота переходу PROCESS B → PROCESS C	0,3909	0,4

Висновок

Універсальний алгоритм підтримує багатоканальне обслуговування і вибір маршруту як за пріоритетом, так і за ймовірністю. У моделі для перевірки вимоги, які PROCESS A не може прийняти, ідуть в обхід, тому для нього це система M/M/2/4 з втратами, і її характеристики відомі точно. Частка вимог, не прийнятих у PROCESS A, становить 0,1241 при теоретичному значенні 0,1240; завантаження і середня довжина черги також збігаються з теорією (таблиця 2). Частота переходу з PROCESS B у PROCESS C дорівнює 0,3909 при заданій ймовірності 0,4.

Баланс вимог виконується: створено 100027, з моделі вийшло 99681, втрачено 344, у системі залишилось 2. Отже, алгоритм працює коректно і може використовуватись для побудови моделей конкретних систем.

Завдання 2

Формалізована модель

Банк подано як мережу масового обслуговування з двох одноканальних систем з обмеженими чергами (рисунок 3). Вимогою є автомобіль клієнта. Параметри елементів наведено в таблиці 3.

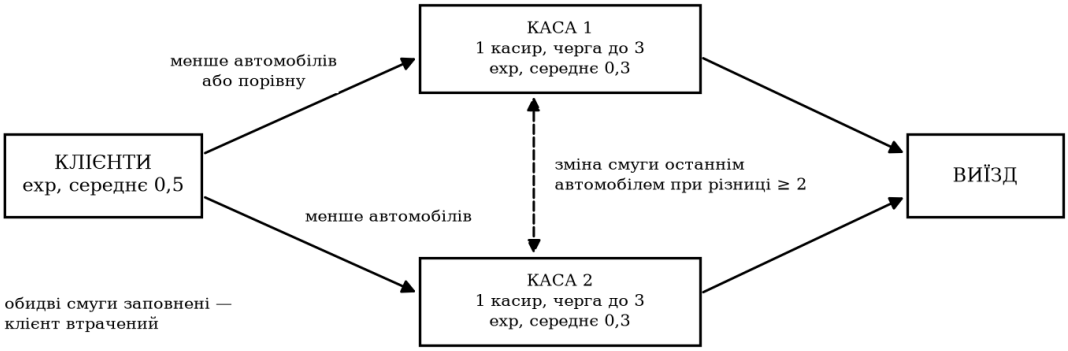


Рисунок 3 – Формалізована модель банку

Таблиця 3 – Елементи моделі банку

Елемент	Клас	Параметри
КЛІЄНТИ	Create	Інтервал надходження експоненційний із середнім 0,5; перше надходження в момент 0,1. Маршрут за правилом: смуга з меншою кількістю автомобілів, при рівності — перша
КАСА 1, КАСА 2	Process	Один пристрій (касир); обслуговування експоненційне із середнім 0,3; черга не більше 3. Початковий стан: касир зайнятий, тривалість обслуговування нормальна (1; 0,3), у черзі 2 автомобілі
ВИЇЗД	Dispose	Підраховує обслужених клієнтів, час їх перебування в банку та інтервали між від'їздами
Зміна смуги	правило моделі	Після кожної події: якщо на одній смузі щонайменше на 2 автомобілі більше, ніж на іншій, останній автомобіль з її черги переїжджає на іншу смугу

При формалізації прийнято такі припущення. Довжина черги на смузі розуміється як кількість автомобілів на смузі разом із тим, що обслуговується: інакше клієнт ставав би в чергу до зайнятого касира при вільному другому, що суперечить змісту задачі. Клієнт вважається втраченим, якщо обидві смуги заповнені (касир зайнятий і в черзі три автомобілі); у цьому разі відмова

фіксується на першій смузі. Час моделювання умовою не заданий, тому взято T = 1000 од. часу, за який у банк прибуває близько двох тисяч клієнтів.

Вихідні характеристики обчислюються так: завантаження касира і середня довжина черги — як середні за часом; середнє число клієнтів у банку — як сума середніх довжин черг і середньої кількості зайнятих касирів; середній інтервал між від'їздами — як середній інтервал між надходженнями вимог в елемент ВИЇЗД; середній час перебування — як середнє різниці між моментом виїзду і моментом прибуття клієнта; відсоток відмов — як відношення кількості втрачених клієнтів до кількості тих, що прибули.

Лістинг файлів

Файл task2_bank.py:

```
"""Завдання 2: банк для автомобілістів — дві каси з окремими смугами, зміна смуги."""
import fun_rand
from create import Create
from dispose import Dispose
from element import Customer
from model import Model
from process import Process
from theory import mmck


ARRIVAL, SERVICE, MAX_QUEUE, TIME, LONG_TIME = 0.5, 0.3, 3, 1000, 100_000



def cars(lane):
    """Кількість автомобілів на смузі: у черзі та біля віконця."""
    return len(lane.queue) + lane.busy



def run(time=TIME, verbose_events=0):
    fun_rand.seed(1)
    create = Create("КЛІЄНТИ", ARRIVAL)
    lanes = [Process(f"КАСА {i}", SERVICE, max_queue=MAX_QUEUE) for i in (1, 2)]
    dispose = Dispose("ВИЇЗД")


    create.set_router(lambda customer: min(lanes, key=cars))  # коротша смуга, при рівності — перша
    create.tnext = 0.1
    for lane in lanes:
        lane.set_next(dispose)
        # початкові умови: касир зайнятий, у черзі два автомобілі
        lane.start_service(Customer(0, 0.0), fun_rand.norm(1.0, 0.3))
        lane.queue += [Customer(0, 0.0), Customer(0, 0.0)]


    changes = [0]


    def change_lane():
        """Останній автомобіль переїжджає на смугу, де щонайменше на два автомобілі менше."""
        for source, target in (lanes, lanes[::-1]):
            while source.queue and cars(source) - cars(target) >= 2:
                target.in_act(source.queue.pop())
                changes[0] += 1
```

```
    model = Model([create, *lanes, dispose], after_event=change_lane)
    model.simulate(time, verbose_events)
    return create, lanes, dispose, changes[0]



def in_bank(lanes, time):
    return sum(lane.queue_area + lane.busy_area for lane in lanes) / time



def main():
    create, lanes, dispose, changes = run(verbose_events=5)
    lost = sum(lane.failure for lane in lanes)
    left = sum(cars(lane) for lane in lanes)


    print(f"\n------------- РЕЗУЛЬТАТИ (T = {TIME}) -------------")
    print(f"Прибуло клієнтів: {create.quantity}, обслужено: {dispose.quantity}, втрачено: {lost}")
    for number, lane in enumerate(lanes, 1):
        print(f"1) середнє завантаження касира {number}:        {lane.mean_load(TIME):.4f}")
    print(f"2) середнє число клієнтів у банку:         {in_bank(lanes, TIME):.4f}")
    print(f"3) середній інтервал між від'їздами:       {dispose.mean_interval():.4f}")
    print(f"4) середній час перебування в банку:       {dispose.mean_time():.4f}")
    for number, lane in enumerate(lanes, 1):
        print(f"5) середнє число клієнтів у черзі {number}:      {lane.mean_queue(TIME):.4f}")
    print(f"6) відсоток клієнтів, яким відмовлено:     {100 * lost / create.quantity:.2f} %")
    print(f"7) число змін під'їзних смуг:              {changes}")


    balance = create.quantity + 6 == dispose.quantity + lost + left
    print(f"\nБаланс: прибуло {create.quantity} + 6 початкових = обслужено {dispose.quantity} "
          f"+ втрачено {lost} + у банку {left}: {'так' if balance else 'НІ'}")


    # завдяки зміні смуг касир не простоює, доки хтось чекає, тому банк загалом — система M/M/2/8
    create, lanes, dispose, _ = run(LONG_TIME)
    p_loss, mean_queue, mean_load = mmck(ARRIVAL, SERVICE, 2, 2 * MAX_QUEUE)
    theory_in_bank = mean_queue + 2 * mean_load
    theory_time = theory_in_bank * ARRIVAL / (1 - p_loss)
    total_load = sum(lane.mean_load(LONG_TIME) for lane in lanes)
    mean_in_bank = in_bank(lanes, LONG_TIME)
    print(f"\nПеревірка на тривалому інтервалі (T = {LONG_TIME}), імітація / теорія M/M/2/8:")
    print(f"  середнє число клієнтів у банку:   {mean_in_bank:.4f} / {theory_in_bank:.4f}")
    print(f"  середній час перебування в банку: {dispose.mean_time():.4f} / {theory_time:.4f}")
    print(f"  сумарне завантаження касирів:     {total_load:.4f} / {2 * mean_load:.4f}")



if __name__ == "__main__":
    main()
```

Результат роботи

```
Подія в КЛІЄНТИ, t = 0.1000
  КЛІЄНТИ: quantity = 1, tnext = 0.8215
  КАСА 1: busy = 1/1, queue = 3, quantity = 0, failure = 0, tnext = 1.3865
  КАСА 2: busy = 1/1, queue = 2, quantity = 0, failure = 0, tnext = 1.4348
  ВИЇЗД: quantity = 0, tnext = inf


Подія в КЛІЄНТИ, t = 0.8215
  КЛІЄНТИ: quantity = 2, tnext = 0.9687
  КАСА 1: busy = 1/1, queue = 3, quantity = 0, failure = 0, tnext = 1.3865
  КАСА 2: busy = 1/1, queue = 3, quantity = 0, failure = 0, tnext = 1.4348
  ВИЇЗД: quantity = 0, tnext = inf


Подія в КЛІЄНТИ, t = 0.9687
  КЛІЄНТИ: quantity = 3, tnext = 1.3107
  КАСА 1: busy = 1/1, queue = 3, quantity = 0, failure = 1, tnext = 1.3865
  КАСА 2: busy = 1/1, queue = 3, quantity = 0, failure = 0, tnext = 1.4348
  ВИЇЗД: quantity = 0, tnext = inf


Подія в КЛІЄНТИ, t = 1.3107
  КЛІЄНТИ: quantity = 4, tnext = 1.6092
```

```
  КАСА 1: busy = 1/1, queue = 3, quantity = 0, failure = 2, tnext = 1.3865
  КАСА 2: busy = 1/1, queue = 3, quantity = 0, failure = 0, tnext = 1.4348
  ВИЇЗД: quantity = 0, tnext = inf


Подія в КАСА 1, t = 1.3865
  КЛІЄНТИ: quantity = 4, tnext = 1.6092
  КАСА 1: busy = 1/1, queue = 2, quantity = 1, failure = 2, tnext = 1.7028
  КАСА 2: busy = 1/1, queue = 3, quantity = 0, failure = 0, tnext = 1.4348
  ВИЇЗД: quantity = 1, tnext = inf


------------ РЕЗУЛЬТАТИ (T = 1000) -------------
Прибуло клієнтів: 2008, обслужено: 2011, втрачено: 2
1) середнє завантаження касира 1:        0.4068
1) середнє завантаження касира 2:        0.2024
2) середнє число клієнтів у банку:         0.6749
3) середній інтервал між від'їздами:       0.4968
4) середній час перебування в банку:       0.3355
5) середнє число клієнтів у черзі 1:      0.0487
5) середнє число клієнтів у черзі 2:      0.0170
6) відсоток клієнтів, яким відмовлено:     0.10 %
7) число змін під'їзних смуг:              127


Баланс: прибуло 2008 + 6 початкових = обслужено 2011 + втрачено 2 + у банку 1: так


Перевірка на тривалому інтервалі (T = 100000), імітація / теорія M/M/2/8:
  середнє число клієнтів у банку:   0.6582 / 0.6591
  середній час перебування в банку: 0.3296 / 0.3296
  сумарне завантаження касирів:     0.5983 / 0.6000
```

Таблиця 4 – Результати моделювання банку (T = 1000)

№	Показник	Значення
1	Середнє завантаження касира 1	0,4068
1	Середнє завантаження касира 2	0,2024
2	Середнє число клієнтів у банку	0,6749
3	Середній інтервал часу між від’їздами клієнтів	0,4968
4	Середній час перебування клієнта в банку	0,3355
5	Середнє число клієнтів у черзі 1	0,0487
5	Середнє число клієнтів у черзі 2	0,0170
6	Відсоток клієнтів, яким відмовлено в обслуговуванні	0,10 %
7	Число змін під’їзних смуг	127

Таблиця 5 – Перевірка моделі банку на тривалому інтервалі (T = 100000)

Показник	Імітація	Теорія M/M/2/8
Середнє число клієнтів у банку	0,6582	0,6591
Середній час перебування клієнта в банку	0,3296	0,3296
Сумарне завантаження касирів	0,5983	0,6000

Висновок

З протоколу видно, що початкові умови відтворено правильно: у момент прибуття першого клієнта обидва касири зайняті, у чергах по два автомобілі.

Перший клієнт стає на першу смугу (на смугах порівну), другий — на другу, а третій і четвертий отримують відмову, бо банк заповнений. За час моделювання прибуло 2008 клієнтів, обслужено 2011, втрачено 2; баланс з урахуванням шести початкових автомобілів виконується.

Касир першої смуги завантажений удвічі більше за касира другої (0,4068 проти 0,2024), оскільки при рівній кількості автомобілів клієнти вибирають першу смугу. Сумарне завантаження близьке до 0,6 — відношення середнього часу обслуговування до середнього інтервалу надходження. У середньому в банку перебуває 0,67 клієнта, клієнт проводить у банку 0,34 од. часу, тобто черги майже немає. Середній інтервал між від'їздами 0,50 практично дорівнює середньому інтервалу надходження, бо майже всі клієнти обслуговуються. Відмову отримали 0,10 % клієнтів, причому всі відмови припали на початок моделювання, коли банк був заповнений за початковими умовами. Смугу змінювали 127 разів.

Завдяки зміні смуг касир ніколи не простоює, доки на сусідній смузі хтось чекає, тому загальна кількість автомобілів у банку поводиться як у двоканальній системі M/M/2/8. На тривалому інтервалі результати імітації збігаються з аналітичними значеннями для цієї системи (таблиця 5), що підтверджує правильність моделі.

Завдання 3

Формалізована модель

Процес прийому хворих подано як мережу масового обслуговування (рисунок 4). Вимогою є хворий; тип хворого призначається при надходженні відповідно до відносних частот 0,5, 0,1 і 0,4. Параметри елементів наведено в таблиці 6.

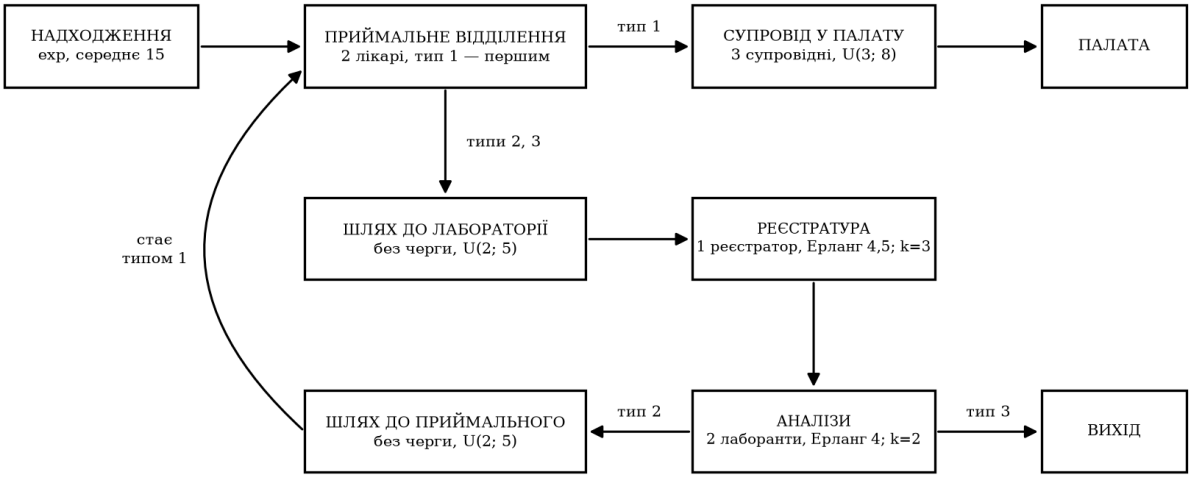


Рисунок 4 – Формалізована модель приймального відділення лікарні

Таблиця 6 – Елементи моделі лікарні

Елемент	Пристроїв	Часова затримка, хв	Черга і маршрут
НАДХОДЖЕННЯ	—	Експоненційна, середнє 15	Призначає тип хворого; далі приймальне відділення
ПРИЙМАЛЬНЕ ВІДДІЛЕННЯ	2 лікарі	Експоненційна, середнє за типом: 15, 40, 30	Черга необмежена, хворі типу 1 обслуговуються першими. Тип 1 — на супровід у палату, типи 2 і 3 — у лабораторію
СУПРОВІД У ПАЛАТУ	3 супровідні	Рівномірна від 3 до 8	Черга необмежена (очікування в приймальному відділенні); далі палата
ШЛЯХ ДО ЛАБОРАТОРІЇ	необмежено	Рівномірна від 2 до 5	Без черги; далі реєстратура
РЕЄСТРАТУРА ЛАБОРАТОРІЇ	1	Ерланга, середнє 4,5, k = 3	Черга необмежена; далі аналізи
АНАЛІЗИ	2 лаборанти	Ерланга, середнє 4, k = 2	Черга необмежена (кімната очікування). Тип 3 — вихід; тип 2 — назад у приймальне відділення і стає типом 1
ШЛЯХ ДО ПРИЙМАЛЬНОГО	необмежено	Рівномірна від 2 до 5	Без черги; далі приймальне відділення
ПАЛАТА, ВИХІД	—	—	Фіксують завершення і час перебування хворого в системі

При формалізації прийнято такі припущення. Час реєстрації в приймальному відділенні розподілений експоненційно із заданим середнім, оскільки закон розподілу в умові не вказано. У реєстратурі лабораторії працює один реєстратор. Супровідний зайнятий лише на час слідування з хворим у палату. Переходи між приймальним відділенням і лабораторією подано елементами з необмеженою кількістю пристроїв: хворі йдуть без супроводу і не чекають один одного. Хворий типу 2 після аналізів повторно проходить приймальне відділення вже як хворий типу 1: з пріоритетом у черзі та середнім часом реєстрації 15 хвилин. Час моделювання T = 100000 хвилин.

Лістинг файлів

Файл task3_hospital.py:

```
"""Завдання 3: приймальне відділення лікарні — три типи хворих, супровідні, лабораторія."""
import math


import fun_rand
from create import Create
from dispose import Dispose
from model import Model
from process import Process


ARRIVAL, TIME = 15, 100_000
REGISTRATION = {1: 15, 2: 40, 3: 30}  # середній час реєстрації за типом хворого



def patient_kind():
    r = fun_rand.uniform01()
    return 1 if r < 0.5 else 2 if r < 0.6 else 3



def run(verbose_events=0):
    fun_rand.seed(1)
    create = Create("НАДХОДЖЕННЯ", ARRIVAL, kind=patient_kind)
    reception = Process("ПРИЙМАЛЬНЕ ВІДДІЛЕННЯ", channels=2,
                        distribution=lambda patient: fun_rand.exp(REGISTRATION[patient.kind]),
                        queue_key=lambda patient: patient.kind != 1)  # тип 1 обслуговується першим
    escort = Process("СУПРОВІД У ПАЛАТУ", 3, 8, "unif", channels=3)
    to_lab = Process("ШЛЯХ ДО ЛАБОРАТОРІЇ", 2, 5, "unif", channels=math.inf)
    registry = Process("РЕЄСТРАТУРА ЛАБОРАТОРІЇ", 4.5, 3, "erlang")
    analysis = Process("АНАЛІЗИ", 4, 2, "erlang", channels=2)
    to_reception = Process("ШЛЯХ ДО ПРИЙМАЛЬНОГО", 2, 5, "unif", channels=math.inf)
    ward = Dispose("ПАЛАТА")
    exit_ = Dispose("ВИХІД З ЛАБОРАТОРІЇ")


    def after_analysis(patient):
        if patient.origin == 3:
            return exit_
        patient.kind = 1  # хворий типу 2 після аналізів розглядається як хворий типу 1
        return to_reception


    create.set_next(reception)
    reception.set_router(lambda patient: escort if patient.kind == 1 else to_lab)
    escort.set_next(ward)
```

```
    to_lab.set_next(registry)
    registry.set_next(analysis)
    analysis.set_router(after_analysis)
    to_reception.set_next(reception)


    processes = [reception, escort, to_lab, registry, analysis, to_reception]
    model = Model([create, *processes, ward, exit_])
    model.simulate(TIME, verbose_events)
    return model, create, processes, ward, exit_



def main():
    model, create, processes, ward, exit_ = run(verbose_events=4)
    reception, _, _, registry, _, _ = processes
    model.print_result()


    total = ward.quantity + exit_.quantity
    overall = (ward.mean_time() * ward.quantity + exit_.mean_time() * exit_.quantity) / total
    lab_interval = registry.mean_interval()
    print("\nСередній час перебування хворого в системі, хв:")
    print(f"  тип 1 (до доставки в палату):     {ward.mean_time(1):.2f}")
    print(f"  тип 2 (до доставки в палату):     {ward.mean_time(2):.2f}")
    print(f"  тип 3 (до виходу з лабораторії):  {exit_.mean_time(3):.2f}")
    print(f"  усі хворі:                        {overall:.2f}")
    print(f"Середній інтервал між прибуттями хворих у лабораторію, хв: {lab_interval:.2f}")


    # робота лікарів на одного хворого: реєстрація за типом і повторна реєстрація хворих типу 2
    work = (0.5 * REGISTRATION[1] + 0.1 * REGISTRATION[2] + 0.4 * REGISTRATION[3]
            + 0.1 * REGISTRATION[1])
    lab_share = registry.arrived / create.quantity
    in_system = sum(len(p.queue) + p.busy for p in processes)
    balance = create.quantity == total + in_system
    print("\nПеревірка (імітація / очікуване значення):")
    print(f"  частка хворих, що потрапили в лабораторію: {lab_share:.4f} / 0.5")
    print(f"  інтервал між прибуттями в лабораторію:     {lab_interval:.2f} / {ARRIVAL / 0.5:.2f}")
    print(f"  завантаження лікарів приймального:         "
          f"{reception.mean_load(TIME):.4f} / {work / ARRIVAL / 2:.4f}")
    print(f"  баланс: надійшло {create.quantity} = у палатах {ward.quantity} "
          f"+ вийшло {exit_.quantity} + у системі {in_system}: {'так' if balance else 'НІ'}")



if __name__ == "__main__":
    main()
```

Результат роботи

```
Подія в НАДХОДЖЕННЯ, t = 0.0000
  НАДХОДЖЕННЯ: quantity = 1, tnext = 2.1644
  ПРИЙМАЛЬНЕ ВІДДІЛЕННЯ: busy = 1/2, queue = 0, quantity = 0, failure = 0, tnext = 8.8339
  СУПРОВІД У ПАЛАТУ: busy = 0/3, queue = 0, quantity = 0, failure = 0, tnext = inf
  ШЛЯХ ДО ЛАБОРАТОРІЇ: busy = 0/inf, queue = 0, quantity = 0, failure = 0, tnext = inf
  РЕЄСТРАТУРА ЛАБОРАТОРІЇ: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  АНАЛІЗИ: busy = 0/2, queue = 0, quantity = 0, failure = 0, tnext = inf
  ШЛЯХ ДО ПРИЙМАЛЬНОГО: busy = 0/inf, queue = 0, quantity = 0, failure = 0, tnext = inf
  ПАЛАТА: quantity = 0, tnext = inf
  ВИХІД З ЛАБОРАТОРІЇ: quantity = 0, tnext = inf


Подія в НАДХОДЖЕННЯ, t = 2.1644
  НАДХОДЖЕННЯ: quantity = 2, tnext = 12.4252
  ПРИЙМАЛЬНЕ ВІДДІЛЕННЯ: busy = 2/2, queue = 0, quantity = 0, failure = 0, tnext = 8.8339
  СУПРОВІД У ПАЛАТУ: busy = 0/3, queue = 0, quantity = 0, failure = 0, tnext = inf
  ШЛЯХ ДО ЛАБОРАТОРІЇ: busy = 0/inf, queue = 0, quantity = 0, failure = 0, tnext = inf
  РЕЄСТРАТУРА ЛАБОРАТОРІЇ: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  АНАЛІЗИ: busy = 0/2, queue = 0, quantity = 0, failure = 0, tnext = inf
  ШЛЯХ ДО ПРИЙМАЛЬНОГО: busy = 0/inf, queue = 0, quantity = 0, failure = 0, tnext = inf
  ПАЛАТА: quantity = 0, tnext = inf
  ВИХІД З ЛАБОРАТОРІЇ: quantity = 0, tnext = inf


Подія в ПРИЙМАЛЬНЕ ВІДДІЛЕННЯ, t = 8.8339
  НАДХОДЖЕННЯ: quantity = 2, tnext = 12.4252
  ПРИЙМАЛЬНЕ ВІДДІЛЕННЯ: busy = 1/2, queue = 0, quantity = 1, failure = 0, tnext = 25.4832
```

```
  СУПРОВІД У ПАЛАТУ: busy = 0/3, queue = 0, quantity = 0, failure = 0, tnext = inf
  ШЛЯХ ДО ЛАБОРАТОРІЇ: busy = 1/inf, queue = 0, quantity = 0, failure = 0, tnext = 11.1155
  РЕЄСТРАТУРА ЛАБОРАТОРІЇ: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  АНАЛІЗИ: busy = 0/2, queue = 0, quantity = 0, failure = 0, tnext = inf
  ШЛЯХ ДО ПРИЙМАЛЬНОГО: busy = 0/inf, queue = 0, quantity = 0, failure = 0, tnext = inf
  ПАЛАТА: quantity = 0, tnext = inf
  ВИХІД З ЛАБОРАТОРІЇ: quantity = 0, tnext = inf


Подія в ШЛЯХ ДО ЛАБОРАТОРІЇ, t = 11.1155
  НАДХОДЖЕННЯ: quantity = 2, tnext = 12.4252
  ПРИЙМАЛЬНЕ ВІДДІЛЕННЯ: busy = 1/2, queue = 0, quantity = 1, failure = 0, tnext = 25.4832
  СУПРОВІД У ПАЛАТУ: busy = 0/3, queue = 0, quantity = 0, failure = 0, tnext = inf
  ШЛЯХ ДО ЛАБОРАТОРІЇ: busy = 0/inf, queue = 0, quantity = 1, failure = 0, tnext = inf
  РЕЄСТРАТУРА ЛАБОРАТОРІЇ: busy = 1/1, queue = 0, quantity = 0, failure = 0, tnext = 16.8306
  АНАЛІЗИ: busy = 0/2, queue = 0, quantity = 0, failure = 0, tnext = inf
  ШЛЯХ ДО ПРИЙМАЛЬНОГО: busy = 0/inf, queue = 0, quantity = 0, failure = 0, tnext = inf
  ПАЛАТА: quantity = 0, tnext = inf
  ВИХІД З ЛАБОРАТОРІЇ: quantity = 0, tnext = inf


------------ РЕЗУЛЬТАТИ (T = 100000) -------------
НАДХОДЖЕННЯ: quantity = 6691
ПРИЙМАЛЬНЕ ВІДДІЛЕННЯ: arrived = 7392, quantity = 7389, failure = 0
    mean queue = 4.2363, mean load = 0.8431, failure probability = 0.0000
СУПРОВІД У ПАЛАТУ: arrived = 4066, quantity = 4066, failure = 0
    mean queue = 0.0003, mean load = 0.0745, failure probability = 0.0000
ШЛЯХ ДО ЛАБОРАТОРІЇ: arrived = 3323, quantity = 3323, failure = 0
    mean queue = 0.0000, mean load = 0.1161, failure probability = 0.0000
РЕЄСТРАТУРА ЛАБОРАТОРІЇ: arrived = 3323, quantity = 3323, failure = 0
    mean queue = 0.0182, mean load = 0.1519, failure probability = 0.0000
АНАЛІЗИ: arrived = 3323, quantity = 3323, failure = 0
    mean queue = 0.0002, mean load = 0.0662, failure probability = 0.0000
ШЛЯХ ДО ПРИЙМАЛЬНОГО: arrived = 701, quantity = 701, failure = 0
    mean queue = 0.0000, mean load = 0.0245, failure probability = 0.0000
ПАЛАТА: quantity = 4066
ВИХІД З ЛАБОРАТОРІЇ: quantity = 2622


Середній час перебування хворого в системі, хв:
  тип 1 (до доставки в палату):     36.35
  тип 2 (до доставки в палату):     199.51
  тип 3 (до виходу з лабораторії):  151.30
  усі хворі:                        98.52
Середній інтервал між прибуттями хворих у лабораторію, хв: 30.09


Перевірка (імітація / очікуване значення):
  частка хворих, що потрапили в лабораторію: 0.4966 / 0.5
  інтервал між прибуттями в лабораторію:     30.09 / 30.00
  завантаження лікарів приймального:         0.8431 / 0.8333
  баланс: надійшло 6691 = у палатах 4066 + вийшло 2622 + у системі 3: так
```

Таблиця 7 – Результати моделювання лікарні (T = 100000 хв)

Показник	Кількість хворих	Значення, хв
Час у системі хворого типу 1 (до доставки в палату)	3365	36,35
Час у системі хворого типу 2 (до доставки в палату)	701	199,51
Час у системі хворого типу 3 (до виходу з лабораторії)	2622	151,30
Час у системі в середньому для всіх хворих	6688	98,52
Інтервал між прибуттями хворих у лабораторію	—	30,09

Таблиця 8 – Характеристики елементів моделі лікарні

Елемент	Пристроїв	Надійшло	Обслужено	Завантаження	Середня черга
ПРИЙМАЛЬНЕ ВІДДІЛЕННЯ	2	7392	7389	0,8431	4,2363
СУПРОВІД У ПАЛАТУ	3	4066	4066	0,0745	0,0003
ШЛЯХ ДО ЛАБОРАТОРІЇ	∞	3323	3323	0,1161	0,0000
РЕЄСТРАТУРА ЛАБОРАТОРІЇ	1	3323	3323	0,1519	0,0182
АНАЛІЗИ	2	3323	3323	0,0662	0,0002
ШЛЯХ ДО ПРИЙМАЛЬНОГО	∞	701	701	0,0245	0,0000

Для елементів з необмеженою кількістю пристроїв у стовпці «Завантаження» наведено середню кількість хворих, що одночасно перебувають у дорозі.

Таблиця 9 – Перевірка моделі лікарні

Показник	Імітація	Очікуване значення
Частка хворих, що потрапили в лабораторію	0,4966	0,5
Інтервал між прибуттями в лабораторію, хв	30,09	30,00
Завантаження лікарів приймального відділення	0,8431	0,8333

Висновок

За час моделювання надійшов 6691 хворий: 4066 доставлено в палати, 2622 залишили лікарню після обстеження, 3 залишились у системі; баланс виконується. Середній час перебування в системі становить 36,3 хв для хворих типу 1, 199,5 хв для типу 2 і 151,3 хв для типу 3; у середньому для всіх хворих — 98,5 хв. Середній інтервал між прибуттями хворих у лабораторію дорівнює 30,09 хв.

«Вузьким місцем» системи є приймальне відділення: лікарі завантажені на 84 %, а в черзі в середньому чекає 4,2 хворого. Решта ресурсів має великий запас: супровідні завантажені на 7 %, реєстратура лабораторії — на 15 %, лаборанти — на 7 %. Хворі типів 2 і 3 проводять у системі в кілька разів більше часу, ніж хворі типу 1, оскільки лікар завжди спочатку вибирає з черги хворих типу 1; хворі типу 2 до того ж проходять приймальне відділення двічі.

Перевірка підтверджує правильність моделі. У лабораторію потрапляє 0,4966 усіх хворих при очікуваній частці 0,5 (типи 2 і 3), тому середній інтервал

між їх прибуттями 30,09 хв удвічі більший за інтервал надходження в лікарню. Завантаження лікарів 0,8431 близьке до розрахункового 0,8333, отриманого як добуток інтенсивності надходження на середній сумарний час реєстрації одного хворого, поділений на кількість лікарів.

Висновок

У ході виконання комп'ютерного практикуму побудовано універсальний алгоритм імітації мереж масового обслуговування і з його допомогою реалізовано моделі двох систем, заданих текстовим описом.

1. Універсальний алгоритм підтримує багатоканальне обслуговування, вибір маршруту за пріоритетом, за ймовірністю або за довільним правилом, пріоритет у черзі, вимоги різних типів і часову затримку, що залежить від типу вимоги. Правильність алгоритму підтверджено збігом з аналітичною моделлю M/M/2/4.
2. Для банку складено формалізовану модель із двох одноканальних систем з обмеженими чергами, вибором коротшої смуги та зміною смуги; визначено всі сім шуканих характеристик. Модель перевірено за балансом вимог і порівнянням із системою M/M/2/8.
3. Для лікарні складено формалізовану модель із шести систем обслуговування з трьома типами хворих, пріоритетом у черзі та поверненням хворих у приймальне відділення; визначено час перебування хворих у системі за типами та інтервал між прибуттями в лабораторію.

Обидві задачі розв'язано без зміни алгоритму імітації: особливості систем задаються параметрами елементів і правилами маршрутизації. Це показує, що формалізм моделей масового обслуговування разом з універсальним алгоритмом дає змогу швидко будувати моделі різних дискретно-подійних систем.

Посилання

1. Комп’ютерний практикум 3 «Побудова імітаційної моделі системи з використанням формалізму моделі масового обслуговування»: методичні вказівки з дисципліни «Комп’ютерне моделювання дискретно-подійних систем». – КПІ ім. Ігоря Сікорського, 2026.
2. Law A. M., Kelton W. D. Simulation Modeling and Analysis. – 3rd ed. – McGraw-Hill, 2000.
3. Повний код практичної роботи: https://github.com/ip34is/7-sem/tree/main/CMDES/CMDES-lab3